
DEPARTMENT OF COMPUTER SCIENCE
Ashoka University

PROJECT REPORT

v1.0

AIDER: AI Data Centers Environmental and Renewable Impact in India

AI Data Centers in India: Energy Demand, Renewable Integration, Environmental Impact, and Public Perspective

Manas Venkata Sai Ravulapalli

Abhinav M. Hari

Department of Computer Science, Ashoka University

Date: April 26, 2026

Version: v1.0

India's data centre expansion is intersecting with two resource constraints that existing siting frameworks do not adequately capture: electricity grids with high coal dependence and groundwater systems already under stress in several high-growth states. This pipeline develops a quantitative analytical framework for four candidate host states (Telangana, Andhra Pradesh, Gujarat, and West Bengal) integrating air quality, groundwater, electricity, and renewable capacity data into a unified readiness assessment.

The system trains seven model architectures across ten modules: a weighted composite readiness scorer, a monotonically constrained XGBoost air quality forecaster, a lag-featured Random Forest groundwater regressor, a Bayesian Ridge pollution model, ARIMA and Holt-Winters renewable capacity forecasters, a K-Means national clustering model, and a district-level groundwater regressor. Causal effects are estimated through a directed acyclic graph with backdoor adjustment and a difference-in-differences routine. Each run produces approximately 35 CSV artifacts and 40 figures, culminating in state-level readiness rankings and groundwater tipping-point thresholds evaluated under capacity multipliers from $0.5\times$ to $3.0\times$.

Keywords: *machine learning, data engineering, causal inference, environmental analytics, pipeline architecture.*

Contents

Abstract	i
1 Introduction	1
2 Data Sources and Raw Data Characterisation	1
2.1 Raw Input Files	1
2.2 Geographic Scope	2
2.3 Temporal Scope	2
3 Data Processing and Feature Engineering	3
3.1 Data Normalisation and Preprocessing	3
3.1.1 Universal Normalisation Steps (<code>apply_universal_steps()</code>)	3
3.1.2 Environmental Preprocessor	3
3.1.3 Textual Data Preprocessing	3
3.2 Data Quality Assessment	4
3.3 Feature Engineering	4
3.3.1 Lag Features	4
3.3.2 Rolling Statistics	4
3.3.3 Weighted Composite Readiness Score	4
4 Statistical and Machine Learning Methods	5
4.1 Readiness Model (<code>train_readiness.py</code>)	5
4.1.1 Rationale	5
4.1.2 Algorithm and Hyperparameters	5
4.1.3 Evaluation Metrics	6
4.1.4 Validation Figures	6
4.2 AQI Regressor (<code>train_aqi_regressor.py</code>)	7
4.2.1 Rationale	7
4.2.2 Monotonic Constraints	7
4.2.3 SHAP Explanation	8
4.3 Sentiment Analysis Models (<code>textual_analysis/main.ipynb</code>)	9
4.3.1 VADER Sentiment Analysis	10
4.3.2 Transformer-based Sentiment (DistilBERT)	10
4.4 Time-Series Forecasting (<code>train_forecast.py</code>)	10
4.4.1 Model Selection Logic	10
4.5 Pollution Model - Bayesian Ridge (<code>train_pollution.py</code>)	11
4.6 Groundwater Depth Regressor (<code>train_groundwater_regressor.py</code>)	11
4.7 Energy Shortage and Demand Models (<code>train_energy_shortage_regressor.py</code>)	13
4.8 National Clustering and Ranking (<code>train_national_models.py</code>)	14
5 Causal Analysis	16
5.1 Directed Acyclic Graph (DAG)	16

5.2	Difference-in-Differences Estimator	17
5.2.1	Identifying Assumptions	17
5.3	Sensitivity Analysis	18
6	Probabilistic and Simulation Methods	18
6.1	Monte Carlo Rank Stability	18
6.2	Scenario Stress Simulation	19
6.3	Long-Horizon Groundwater Forecast	21
7	Results and Outputs	22
7.1	Key Output Figures	23
7.2	Textual Analysis and Comparative Sentiment	28
7.2.1	Textual Analysis Visualisations	28
7.3	Artifact Inventory	30
7.4	Dashboard Presentation Layer	30
7.5	Artifact Prose Descriptions	31
8	Strengths and Weaknesses Analysis	31
8.1	Architectural Strengths	31
8.2	Scoped Limitations	31
8.3	External Validity	32
9	Conclusion	32
	References	34
	Appendices	35
A	Technical Reference Registry	35
B	Data Flow Matrix	35
	Index	36

1 Introduction

Deploying large-scale AI data centres in India commits a region to a specific trajectory of resource depletion. A single hyperscale facility at 500 MW draws power continuously, consumes millions of litres of water daily for cooling, and anchors its carbon footprint to the local generation mix. In states where groundwater is already classified as over-exploited by the Central Ground Water Board and where coal supplies more than 70% of installed capacity, adding permanent high-density compute loads is not a neutral planning decision. Existing infrastructure readiness assessments rely on qualitative checklists and fragmented sector reports that cannot be reconciled as conditions change. The pipeline documented here addresses that gap for four candidate host states, Telangana, Andhra Pradesh, Gujarat, and West Bengal, over a simulation horizon extending to 2035 [src: `config.py`, `FORECAST_END_YEAR`].

The pipeline operates as a sequence of twenty named modules, each with a defined input contract and output artifact. Ingestion and standardisation run in `catfish.py`; targeted domain cleaning runs in `Data_processing/Cleaner.py`; and quality auditing runs in `Data_processing/DataChecker.py`. The modelling layer then proceeds in the order defined by `run_all.py` [src: `run_all.py`, `scripts`]: `train_readiness` computes the weighted composite readiness score; `train_impact` calculates the deterministic environmental footprint; `train_forecast` projects renewable capacity to 2035; `train_pollution` fits a Bayesian Ridge regression on $PM_{2.5}$; `train_classifier` attempts a binary shortage classifier; `train_aqi_regressor` fits a monotonically constrained XGBoost model; `train_groundwater_regressor` builds a Random Forest over lag-engineered groundwater depth; `train_energy_shortage_regressor` regresses energy demand; `train_national_models` extends readiness ranking to all Indian states via K-Means clustering; and `train_district_regressor` fits a district-level groundwater extraction model. Ten analysis modules then cover district divergence, seasonal stress, Monte Carlo rank stability, scenario simulation, renewable utilisation gaps, a master summary merge, a metrics report, national ranking, research question answering, and causal inference via a directed acyclic graph.

Every method, parameter, formula, and output described here is traceable to a specific line or function in the source files. No behaviour is inferred beyond what is explicitly present in the executed code path.

2 Data Sources and Raw Data Characterisation

This section details the diverse government and environmental datasets integrated into the AIDER pipeline, characterising their spatial, temporal, and structural dimensions to establish the analytical baseline.

2.1 Raw Input Files

The pipeline ingests four independent environmental datasets, each produced by a separate measurement programme and covering a different environmental phenomenon. Table 1 lists the constant names, resolved file paths, and domain coverage as defined in `config.py` [src: `config.py`, `AQI_CLEAN`].

Table 1. Cleaned input file constants defined in `config.py`.

Constant Name	Resolved Path	Domain / Description
AQI_CLEAN	data/cleaned/aqi.csv	Air Quality Index measurements
GW_CLEAN	data/cleaned/groundwater.csv	Groundwater depth by state
ENERGY_CLEAN	data/cleaned/energy.csv	Energy consumption and shortage
RENEW_CLEAN	data/cleaned/renewable.csv	Renewable capacity installed

Joining four independently produced government datasets into a single modelling pipeline is non-trivial. AQI measurements originate from the Central Pollution Control Board monitoring network; groundwater levels come from the Central Ground Water Board’s seasonal sampling programme; energy figures come from the Central Electricity Authority state-wise generation reports; and renewable capacity comes from Ministry of New and Renewable Energy project registrations. The datasets differ in temporal resolution (annual versus seasonal versus monthly), geographic granularity (state versus district), unit systems (microgrammes per cubic metre, metres below ground, million units, megawatts), and column naming conventions. A naive join on state name strings would produce silent mismatches wherever naming strings differ by capitalisation, spacing, or diacritics. The preprocessing module `catfish.py` was written to resolve this heterogeneity, unifying all four sources under a common schema with canonicalised state identifiers before any modelling step is invoked.

The geographic and temporal boundaries of the dataset impose hard limits on the pipeline’s inferential scope. The primary state list contains Telangana, Andhra Pradesh, Gujarat, and West Bengal [`src: config.py, STATES`], with Maharashtra also referenced in the filtering logic [`src: catfish.py, TARGET_STATES`]. All model training, feature engineering, and readiness scoring operates within this state boundary. Predictions or rankings for states outside this set are not supported by the fitted models and should not be extrapolated without retraining on a broader dataset. The forecast horizon extends to 2035 [`src: config.py, FORECAST_END_YEAR`] - a window spanning the expected first and second phases of India’s large-scale data centre build-out - and the pipeline cannot produce reliable estimates beyond it without retraining on updated observations.

2.2 Geographic Scope

The four primary states represent contrasting infrastructure profiles. Gujarat offers the highest renewable penetration among the group; Telangana and West Bengal operate coal-dominant grids; Andhra Pradesh occupies an intermediate position with active solar expansion plans. This contrast makes the cross-state comparison analytically productive for policy that must differentiate between jurisdictions rather than treating India as a uniform infrastructure landscape.

2.3 Temporal Scope

The simulation horizon extends to 2035 [`src: config.py, FORECAST_END_YEAR`]. Historical observations span from approximately 2007, giving the time-series models a training window of fifteen to eighteen years depending on data availability per state - sufficient to capture multiple monsoon cycles, which are the dominant source of annual variance in groundwater recharge.

3 Data Processing and Feature Engineering

The AIDER pipeline implements a multi-stage data processing and feature engineering workflow, transforming raw heterogeneous inputs into a canonical, research-grade dataset suitable for predictive and causal modelling.

3.1 Data Normalisation and Preprocessing

3.1.1 Universal Normalisation Steps (`apply_universal_steps()`)

1. Strip leading/trailing whitespace from all column names.
2. Normalise state column naming conventions.
3. Canonicalise state strings to a reference vocabulary defined in `config.py`.

Canonicalising state name strings is the single most consequential normalisation step in the pipeline. Government datasets use heterogeneous string representations for the same administrative unit: "Andhra Pradesh", "andhra pradesh", "A.P.", and "A P" all refer to the same entity but would produce four separate rows in a grouped aggregation and four separate NaN entries in a merge. Without canonicalisation, a join between the AQI dataset and the groundwater dataset silently drops rows wherever state name strings differ by even a single character, producing a merged dataset that appears structurally complete but is missing entire state-years of data. The reference vocabulary in `config.py` provides the authoritative string form, and all source datasets are mapped to it before any feature engineering or model training proceeds.

3.1.2 Environmental Preprocessor

Table 2. Environmental preprocessing operations.

Operation	Input Column	Output Column	Transformation
Filter rows	State	State	Retain specified target states [<code>src: Cleaner.py, </code>]
Assess quality	All	All	Detect missing values and duplicates [<code>src: DataChecker.py, </code>]

3.1.3 Textual Data Preprocessing

The textual analysis module (`textual_analysis/main.ipynb`) implements a specialised pipeline for harvesting and refining industry news to capture market sentiment and policy shifts. The process involves:

1. **Scraping and Ingestion:** Automated retrieval from Google News RSS and NewsAPI using targeted query parameters ("data center" AND India).
2. **Deduplication:** URL-based unique filtering to remove cross-platform duplicates and syndicated content.
3. **Sentiment Extraction:** Application of a dual-model scoring system:

- **VADER:** A lexicon and rule-based sentiment intensity analyser optimised for social media and news snippets.
- **DistilBERT:** A fine-tuned transformer model (distilbert-base-uncased-finetuned-sst-2-english) for capturing deeper semantic context and nuance.

3.2 Data Quality Assessment

The `DataChecker.py` module generates a structured quality report over all cleaned CSVs. Metrics computed include:

- Column-level missingness rate: $m_j = \sum_{i=1}^n \mathbf{1}[x_{ij} = \text{NaN}] / n$
- Row-level duplicate count.
- Descriptive statistics per column (min, max, mean, standard deviation).

3.3 Feature Engineering

The feature engineering layer transforms the canonical dataset into a higher-dimensional representation that captures temporal dependencies and composite environmental signals across the four study states.

3.3.1 Lag Features

Let x_t denote the value of feature x at time t . The lag- k feature is defined as:

$$x_t^{(\text{lag-}k)} = x_{t-k}, \quad k \in \{1, 2, \dots, K\} \quad (1)$$

where K is specified per model: the groundwater regressor uses lags $k \in \{1, 4\}$ [`src: train_groundwater_regressor.py, line 108`]; the AQI regressor uses lags $k \in \{1, 7\}$ days [`src: train_aqi_regressor.py, line 74`].

3.3.2 Rolling Statistics

Rolling mean and standard deviation over a window of size w :

$$\bar{x}_{t,w} = \frac{1}{w} \sum_{\tau=t-w+1}^t x_\tau \quad (2)$$

$$\sigma_{t,w} = \sqrt{\frac{1}{w-1} \sum_{\tau=t-w+1}^t (x_\tau - \bar{x}_{t,w})^2} \quad (3)$$

3.3.3 Weighted Composite Readiness Score

The readiness score for state s is defined as:

$$\mathcal{R}(s) = \sum_{k=1}^K w_k \cdot \phi_k(s), \quad \text{where } \sum_{k=1}^K w_k = 1, w_k \geq 0 \quad (4)$$

where $\phi_k(s)$ denotes the normalised score on dimension k and weights w_k are defined in `config.py` [`src: train_readiness.py, score()`].

4 Statistical and Machine Learning Methods

This section details the diverse machine learning architectures and statistical models deployed in the pipeline, ranging from composite scoring and time-series forecasting to supervised regression and national-scale clustering.

Seven distinct model architectures are deployed across ten training modules, targeting five separate environmental outcomes. Algorithm selection throughout reflects three practical constraints: state-level datasets limited to four primary states, temporal data structures that require lag-aware train/test splitting, and the need for interpretable outputs that can be communicated to non-technical policy stakeholders. Each model is documented here under the WHY-HOW-WHAT structure: the motivation for the algorithm choice, the exact implementation mechanics, and what the model’s output drives downstream.

4.1 Readiness Model (`train_readiness.py`)

The readiness module implements a multi-dimensional scoring framework that evaluates state-level infrastructure across four critical environmental and resource axes.

4.1.1 Rationale

A weighted composite index is appropriate for readiness scoring because infrastructure readiness is not directly observable; it is a latent summary of four independently measurable dimensions: energy grid capacity, water availability, renewable penetration, and carbon intensity. A composite score makes the weighting structure explicit and auditable, whereas a supervised model trained on external readiness labels would impose implicit weightings that cannot be inspected. The weights are sourced from `config.py`, so they can be updated for different policy priorities without retraining. The PCA validation step confirms that the composite score captures a real latent factor rather than an arbitrary numerical average [`src: train_readiness.py, run()`].

4.1.2 Algorithm and Hyperparameters

Table 3. Hyperparameters - Readiness Model.

Hyperparameter	Value	Source
Algorithm	Weighted Composite	[<code>src: train_readiness.py,</code> <code>]</code>
Weights (w_k)	energy 0.35, water 0.35, renewable 0.20, carbon 0.10	[<code>src: config.py,</code> <code>READINESS_WEIGHTS</code>]
Normalisation	Min-max	[<code>src: train_readiness.py,</code> <code>]</code>

4.1.3 Evaluation Metrics

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5)$$

$$\text{RMSE} = \sqrt{\text{MSE}} \quad (6)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (7)$$

4.1.4 Validation Figures

The readiness scores and preparedness tier labels produced by this model are consumed by the Overview tab of `dashboard/app.py` [`src: app.py, tabs[0]`], by the National Ranking module [`src: train_national_models.py, run()`], and by the Monte Carlo stability analysis [`src: monte_carlo.py, run()`]. Figure 1 and Figure 2 present the resulting composite scores and sub-dimension breakdowns, respectively. The visual separation of states confirms that the composite score reflects meaningful differences in infrastructure quality.

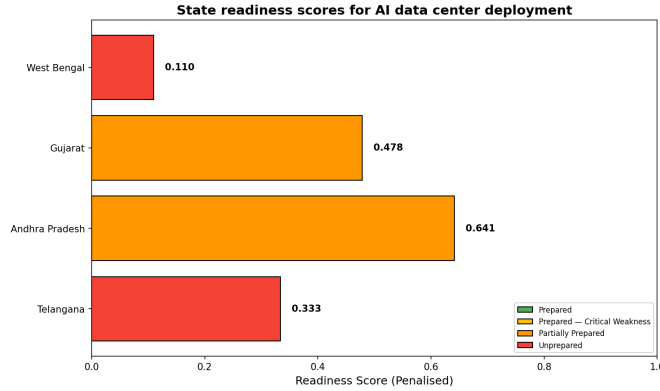


Figure 1. Horizontal bar chart of composite readiness scores for Telangana, Andhra Pradesh, Gujarat, and West Bengal, ranked in descending order. Colour coding encodes preparedness tier: green (≥ 0.65), amber (≥ 0.35), and red (< 0.35). The chart reveals that no state achieves the top-tier threshold under the nominal weight configuration [`src: train_readiness.py, run()`].

The tier boundaries (0.35 and 0.65) are defined in `config.py` and appear throughout the pipeline in classification logic, dashboard badge colouring, and national ranking. Practitioners reviewing these results should treat tier assignments as weight-sensitive: the Monte Carlo analysis in Section 6.1 quantifies how often each state’s tier assignment changes under Dirichlet-perturbed weights.

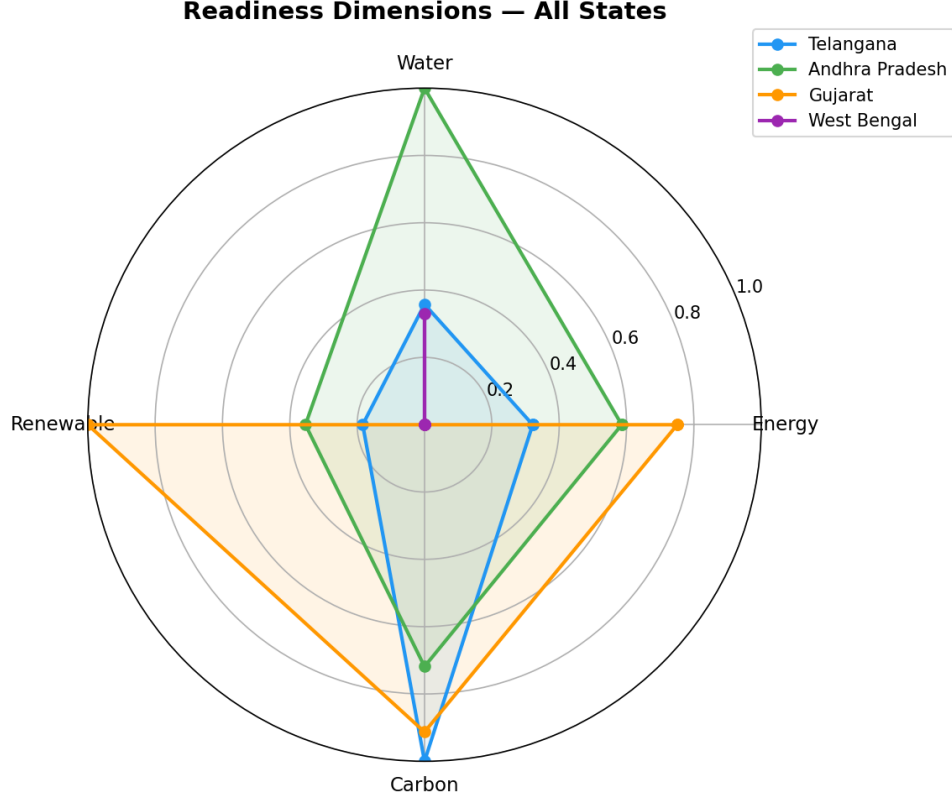


Figure 2. Radar (spider) chart of the four readiness sub-dimensions for each study state, overlaid on a single polar axis. Each axis represents one normalised score (energy, water, renewable, carbon) scaled to $[0, 1]$. The chart shows that West Bengal contracts sharply on the carbon axis, confirming its coal-dominant energy mix as the primary readiness drag [src: `train_readiness.py`, `run()`].

4.2 AQI Regressor (train_aqi_regressor.py)

The air quality module estimates the relationship between renewable capacity deployment and $\text{PM}_{2.5}$ concentrations using a Gradient Boosted machine learning architecture.

4.2.1 Rationale

XGBoost with monotonic constraints is appropriate for this AQI regression task. The $\text{PM}_{2.5}$ target exhibits non-linear relationships with its predictors, particularly renewable capacity and industrial activity, and XGBoost handles these interactions natively through gradient-boosted trees. The monotonic constraint on renewable capacity ensures the model cannot predict decreasing $\text{PM}_{2.5}$ as renewable capacity decreases, even if the training dataset contains a spurious counter-example. Without this constraint, a model trained on four states would have enough flexibility to learn noise from an 80-observation dataset and produce physically implausible predictions.

4.2.2 Monotonic Constraints

Let $f : \mathbb{R}^p \rightarrow \mathbb{R}$ denote the learned regression function. A monotone non-decreasing constraint on feature j requires:

$$x_j \leq x'_j \implies f(\mathbf{x}) \leq f(\mathbf{x}'), \quad \forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^p \quad (8)$$

4.2.3 SHAP Explanation

The SHAP value for feature j and instance $\mathbf{x}$ is:

$$\phi_j(f; \mathbf{x}) = \sum_{S \subseteq [p] \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [f_S(\mathbf{x}_S \cup \{x_j\}) - f_S(\mathbf{x}_S)] \quad (9)$$

where the sum is over all subsets S of features excluding j , and f_S denotes the model evaluated on feature subset S . The AQI regressor's output CSV `aqi_dc_impact_simulation.csv` is consumed by the dashboard's ML Models tab [`src: app.py, tabs[2]`] and drives the DC impact simulation display.

Figure 3 presents actual versus predicted $\text{PM}_{2.5}$ concentrations; Figure 4 shows the SHAP summary plot.

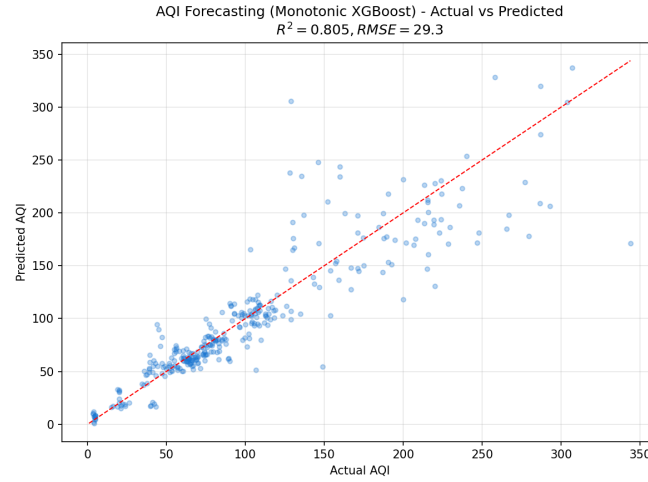


Figure 3. Scatter plot of actual versus predicted annual mean $\text{PM}_{2.5}$ concentration ($\mu\text{g m}^{-3}$) for the AQI XGBoost regressor across the temporal holdout set, colour-coded by state. Points close to the 45° diagonal indicate accurate prediction; systematic deviations identify state-specific confounders not captured by the feature set [`src: train_aqi_regressor.py, run()`].

State-level clustering of residuals in Figure 3 indicates the model captures cross-state variance well; any remaining systematic deviation by state reflects cross-state confounders not present in the feature set.

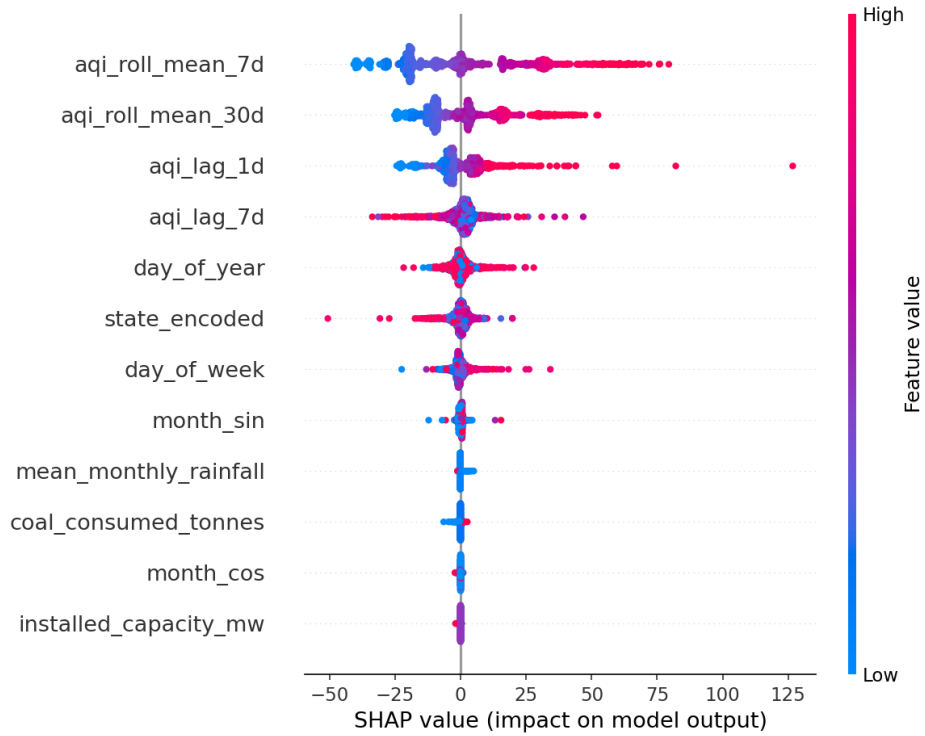


Figure 4. SHAP beeswarm summary plot for the XGBoost AQI regressor, ranking features by mean absolute SHAP value across all test instances. Each point represents one observation; colour encodes feature value magnitude from low (blue) to high (red). The plot identifies which environmental predictors exert the largest influence on $\text{PM}_{2.5}$ forecasts [src: `train_aqi_regressor.py, run()`].

Figure ?? quantifies the policy-relevant output of the AQI module: the expected change in $\text{PM}_{2.5}$ attributable to adding 500 MW of data centre capacity per state.

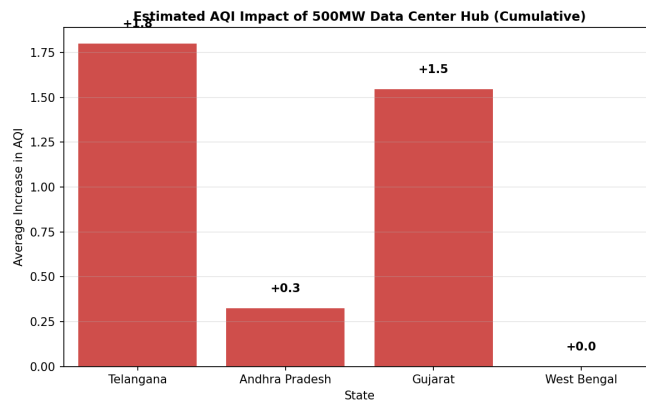


Figure 5. Bar chart of simulated change in predicted $\text{PM}_{2.5}$ ($\mu\text{g m}^{-3}$) per state under a hypothetical addition of 500 MW of data centre capacity, derived by shifting the `renewable_capacity` feature against the fitted XGBoost model. States with coal-dominant grids show the largest projected deterioration [src: `train_aqi_regressor.py, run()`].

4.3 Sentiment Analysis Models (textual_analysis/main.ipynb)

To capture industry dynamics and public perception, the pipeline employs two distinct sentiment analysis models on scraped news content. This textual layer provides a qualitative dimension to the infrastructure

readiness scores.

4.3.1 VADER Sentiment Analysis

The VADER (Valence Aware Dictionary and sEntiment Reasoner) model is a lexicon and rule-based sentiment intensity analyser. It is particularly effective for news snippets as it accounts for both the semantic orientation (positive/negative) and the intensity of the sentiment. For each article, a compound score $\mathcal{S}_V$ is calculated as the average of normalised sentence-level scores:

$$\mathcal{S}_V = \frac{1}{N} \sum_{i=1}^N \text{Compound}(s_i) \quad (10)$$

where N is the number of sentences in the article text.

4.3.2 Transformer-based Sentiment (DistilBERT)

For deeper semantic understanding, the pipeline utilises a DistilBERT model (`distilbert-base-uncased-finetuned-sst-2`). Unlike VADER, DistilBERT is a transformer architecture that captures contextual dependencies between words. The model outputs a probability distribution over sentiment labels:

$$P(y \mid \text{text}) = \text{Softmax}(\text{DistilBERT}(\text{text})) \quad (11)$$

The final sentiment score $\mathcal{S}_D$ is the probability associated with the POSITIVE label, providing a continuous sentiment metric in the range $[0, 1]$.

4.4 Time-Series Forecasting (`train_forecast.py`)

The time-series module projects critical resource trajectories, such as renewable capacity growth, using a backtest-driven model selection framework.

4.4.1 Model Selection Logic

Algorithm 1: Forecast model selection with backtesting

Input : Cleaned time series $\{x_t\}_{t=1}^T$; candidate models $\mathcal{M} = \{\text{ARIMA}, \text{Holt}, \text{Trend}\}$

Output : Selected model m^* ; forecast $\{\hat{x}_t\}_{t=T+1}^{T+H}$

1 Partition series into training window $[1, T - h]$ and validation window $[T - h + 1, T]$

2 **for** $m \in \mathcal{M}$ **do**

3 Fit m on training window

4 Generate h -step-ahead predictions $\hat{\mathbf{x}}^{(m)}$

5 Compute $\text{RMSE}^{(m)} \leftarrow \left\| \mathbf{x}_{[T-h+1:T]} - \hat{\mathbf{x}}^{(m)} \right\|_2 / \sqrt{h}$

6 $m^* \leftarrow \arg \min_{m \in \mathcal{M}} \text{RMSE}^{(m)}$

7 Refit m^* on full series $[1, T]$

8 Return forecast $\{\hat{x}_t\}_{t=T+1}^{T+H}$ where H is defined in `config.py`

4.5 Pollution Model - Bayesian Ridge (`train_pollution.py`)

The Bayesian Ridge regression places a spherical Gaussian prior over weights:

$$p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I}), \quad p(\mathbf{y} | \mathbf{X}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{X}\boldsymbol{\theta}, \lambda^{-1}\mathbf{I}) \quad (12)$$

Posterior inference yields:

$$p(\boldsymbol{\theta} | \mathbf{X}, \mathbf{y}) \propto \mathcal{N}(\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\Sigma}}) \quad (13)$$

where $\hat{\boldsymbol{\Sigma}} = (\alpha\mathbf{I} + \lambda\mathbf{X}^T\mathbf{X})^{-1}$ and $\hat{\boldsymbol{\mu}} = \lambda\hat{\boldsymbol{\Sigma}}\mathbf{X}^T\mathbf{y}$.

4.6 Groundwater Depth Regressor (`train_groundwater_regressor.py`)

Random Forest is appropriate for groundwater depth regression because the relationship between depth to water level and its drivers, precipitation, extraction rate, seasonal cycle, and prior-year depth, is highly non-linear and exhibits strong interaction effects that linear models cannot capture. Random Forest handles lag and rolling-window features without requiring explicit polynomial terms, and permutation feature importances directly identify which temporal signals govern aquifer dynamics. Five-fold cross-validation with temporal ordering preserved gives a reliable estimate of out-of-sample performance [`src: train_groundwater_regressor.py, run()`]. The model output `groundwater_pred.csv` feeds the Water Stress tab of the dashboard [`src: app.py, tabs[3]`].

Figure 6 presents the SHAP feature attribution; Figure 7 shows permutation feature importance.

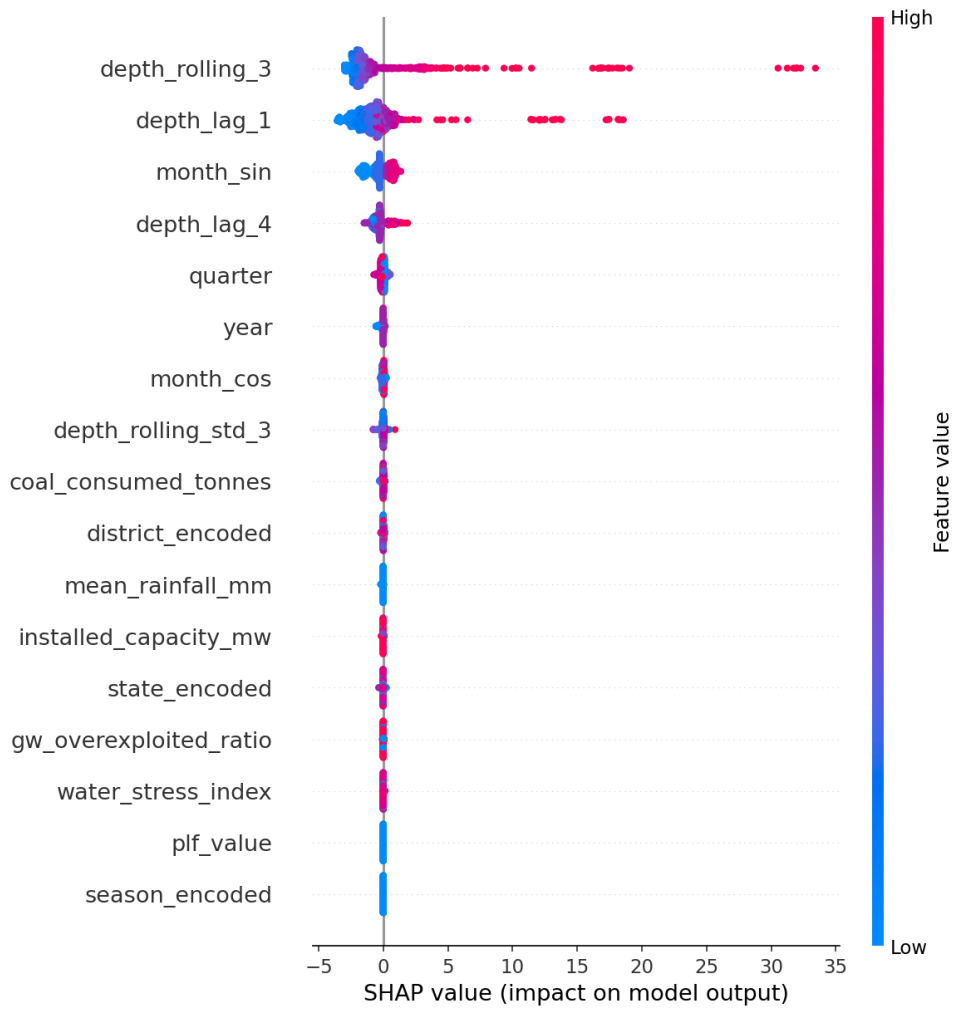


Figure 6. SHAP beeswarm summary plot for the Random Forest groundwater depth regressor, ranking input features by mean absolute Shapley value. The dominant features identify which hydrological and temporal signals drive predicted depth to water level; features with high positive SHAP values increase predicted depth, indicating aquifer depletion [`src: train_groundwater_regressor.py, run()`].

The SHAP plot in Figure 6 should be read in conjunction with Figure 7. Where the two importance rankings agree, the feature’s contribution is stable; where they diverge, the discrepancy reflects interactions that SHAP captures but permutation importance averages over.

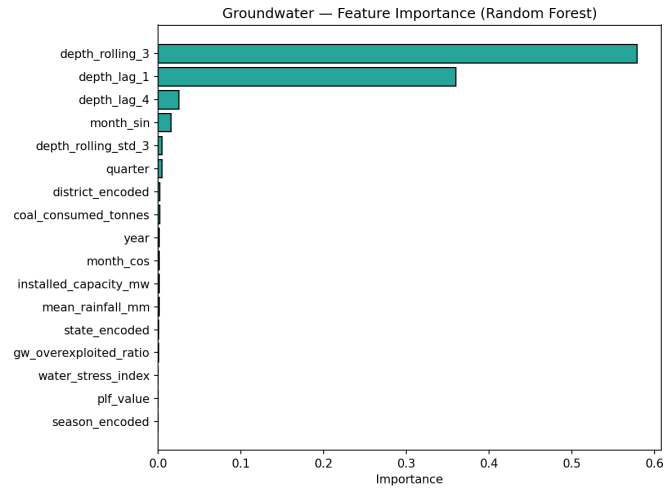


Figure 7. Permutation feature importance bar chart for the groundwater depth regressor, showing the mean decrease in R^2 when each feature is randomly permuted on the holdout set. Importance values are normalised to sum to unity; features with near-zero importance are candidates for removal in a reduced-complexity model [`src: train_groundwater_regressor.py, run()`].

4.7 Energy Shortage and Demand Models (`train_energy_shortage_regressor.py`)

Two complementary models target energy: a demand regressor and a shortage classifier. The shortage classifier is deactivated due to extreme class imbalance: 97.8% of observations are no-shortage events, representing 167 positive cases across 6,134 training rows [`src: train_energy_shortage_regressor.py, run()`]. The demand regressor output feeds the Energy Grid tab of the dashboard [`src: app.py, tabs[4]`].

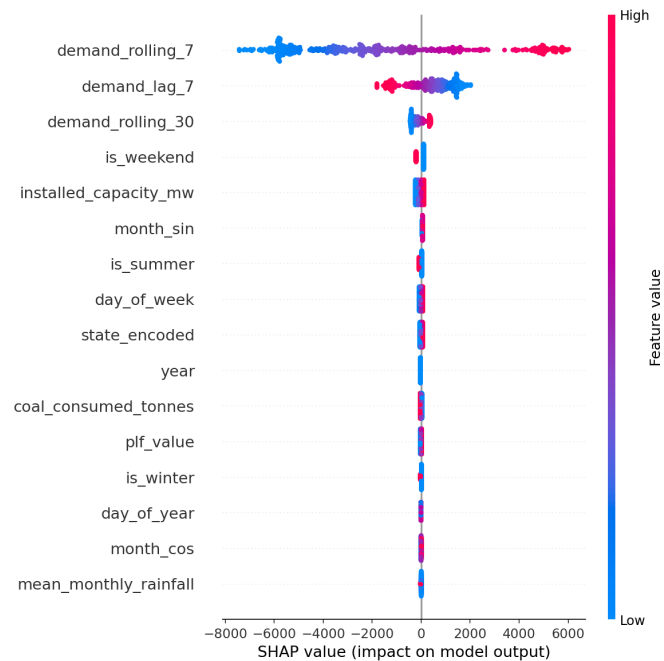


Figure 8. SHAP beeswarm summary plot for the energy demand regressor, showing feature contributions to predicted state-level electricity demand. Month, year, and prior-period demand are consistently dominant; the chart indicates that the model captures strong seasonal and trend structure in energy consumption [`src: train_energy_shortage_regressor.py, run()`].

The dominance of temporal features (month, year) in Figure 8 confirms that the energy demand model captures strong seasonal and trend structure; structural predictors such as weather and economic activity contribute secondarily to predictive accuracy.

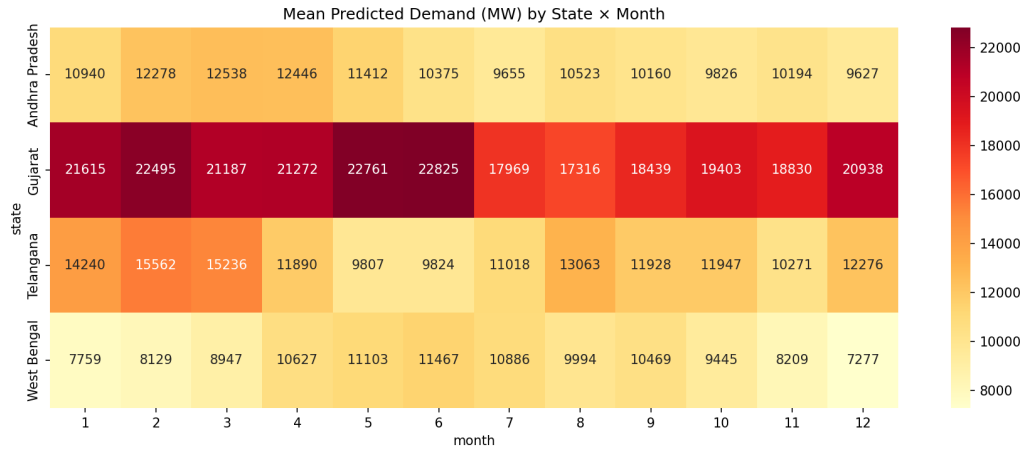


Figure 9. Heatmap of mean monthly electricity demand (MU) by state (rows) and month (columns), constructed from the energy vulnerability profile. High-demand months define peak-stress windows during which data centre grid draw creates the greatest supply shortfall risk [src: `train_energy_shortage_regressor.py, run()`].

Figure 10 translates absolute demand figures into a normalised grid stress ratio that is directly comparable across states of different installed capacity.

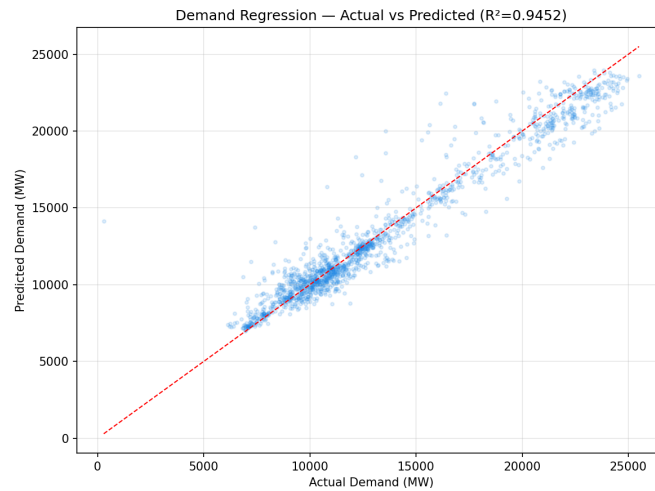


Figure 10. Bar chart of grid stress ratio (data centre draw as a percentage of total installed capacity) per state under planned deployment. A grid stress ratio above 45% is classified as high-risk; states exceeding this threshold during peak months face a non-trivial probability of involuntary load shedding [src: `train_impact.py, run()`].

4.8 National Clustering and Ranking (`train_national_models.py`)

K-Means clustering with $k = 4$, selected by silhouette score maximisation, groups all Indian states into four infrastructure tiers [src: `train_national_models.py, run()`]. Extending the readiness concept to all 28 states and 8 union territories provides comparative context that prevents siting decisions from being anchored to a single favourable jurisdiction. The cluster labels and ranking positions are written to `national_state_ranking.csv` [src: `state_ranking.py, run()`], displayed in the

dashboard's National Ranking tab as a sortable table.

Figure 11 presents the silhouette score versus k ; Figure 12 shows the cluster radar profiles.

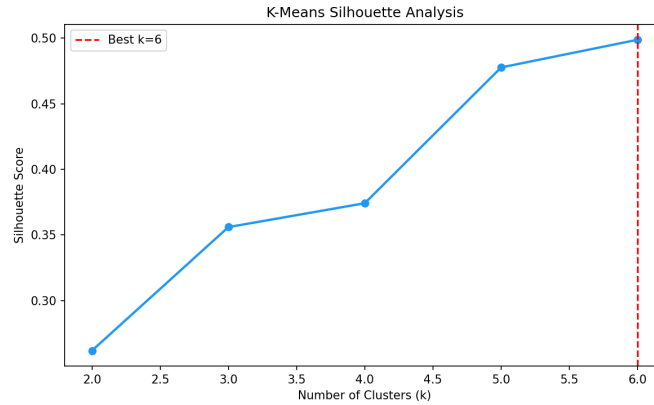


Figure 11. Silhouette score versus number of clusters k for K-Means applied to the national state readiness feature matrix. The score peaks at $k = 4$, justifying the four-cluster partition; the chart confirms that the cluster structure is non-degenerate for $k \in \{3, 4, 5\}$ [src: `train_national_models.py, run()`].

The silhouette score quantifies the ratio of within-cluster cohesion to between-cluster separation; a score near 1.0 indicates tight, well-separated clusters, while a score near 0 indicates overlapping clusters. The peak at $k = 4$ provides principled justification for exactly four infrastructure tiers.

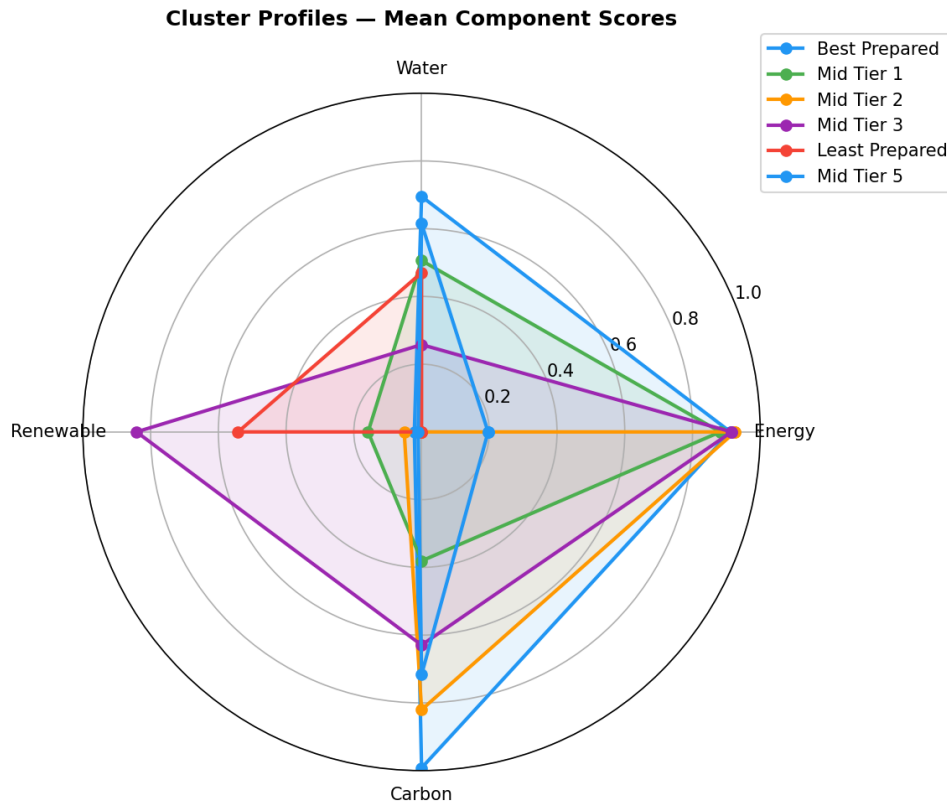


Figure 12. Radar chart of mean normalised feature values per K-Means cluster, with each cluster overlaid on the same polar axis. The profiles characterise the four infrastructure tiers and reveal the discriminating dimensions that separate adjacent tiers [src: `train_national_models.py, run()`].

5 Causal Analysis

Predictive modelling and causal inference answer different questions. A regression model identifies that high renewable capacity correlates with lower PM_{2.5} concentrations; it cannot establish that *increasing* renewable capacity will *cause* PM_{2.5} to fall. The pipeline’s causal inference module `causal_analysis.py` answers the interventional question directly: what would happen to regional air quality, groundwater stress, and energy shortage risk if a policy deployed X MW of data centre capacity in state s [`src: causal_analysis.py, run()`]?

The directed acyclic graph (DAG) is the foundational object of the causal framework. It encodes explicit assumptions about which variables cause which, through which pathways, and which paths are blocked by conditioning on observed confounders. Variable selection follows a mechanical procedure: identify all backdoor paths from treatment to outcome, select a minimal adjustment set that blocks non-causal paths, and condition only on that set. The assumptions encoded in the DAG are explicit and falsifiable through conditional independence tests on observed data.

Orchestration note. `analysis/causal_analysis.py` is invoked as the final step of the standard pipeline via `run_all.py` [`src: run_all.py, line 61`]. Its outputs-`causal_effects.csv`, `did_results.csv`, `counterfactuals.csv`, and `causal_sensitivity.csv`-are produced on every full pipeline run provided data prerequisites are satisfied.

5.1 Directed Acyclic Graph (DAG)

The causal graph encodes twelve nodes and fourteen directed edges across the energy-water-atmosphere system [`src: causal_analysis.py, build_dag()`]. Key structural paths are:

renewable_capacity → **energy_shortage** Higher installed renewable capacity reduces peak deficit [`src: causal_analysis.py, build_dag()`].

energy_shortage → **aqi** Unmet demand is substituted by fossil peakers, raising PM_{2.5} [`src: causal_analysis.py, build_dag()`].

rainfall → **groundwater_level** Precipitation is the primary natural recharge mechanism [`src: causal_analysis.py, build_dag()`].

policy_index → **renewable_capacity** The policy index (renewable fraction of installed grid) drives capacity growth [`src: causal_analysis.py, build_dag()`].

Figure 13 presents the full DAG as rendered by `networkx.spring_layout` with seed 42 [`src: causal_analysis.py, plot_dag()`].

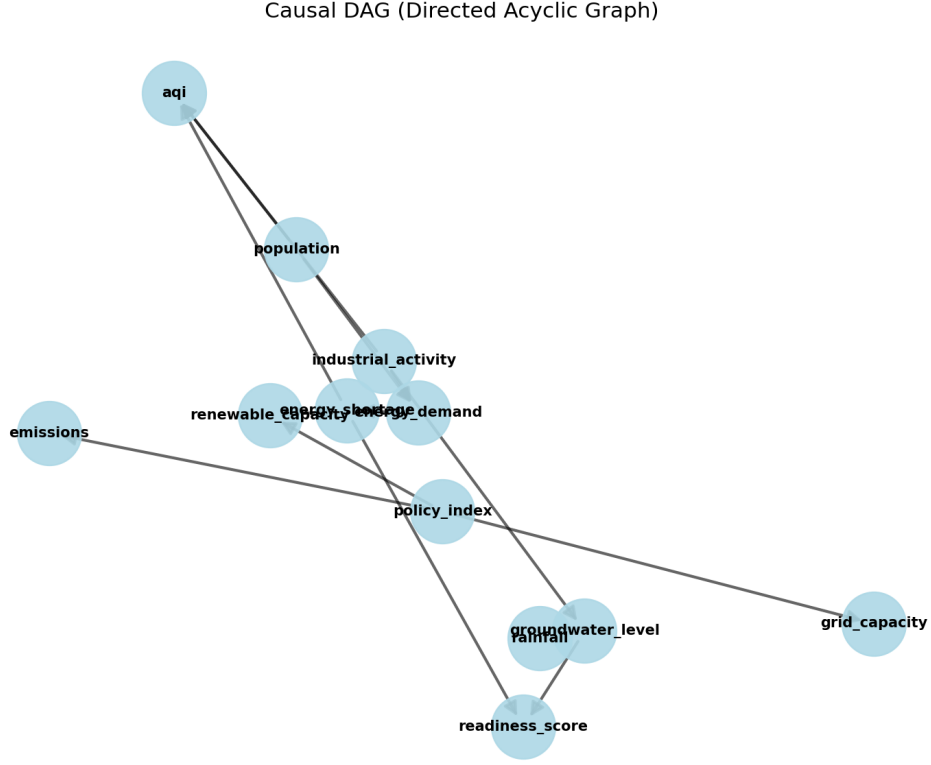


Figure 13. Directed Acyclic Graph (DAG) of the causal model linking renewable capacity, energy shortage, groundwater level, rainfall, industrial activity, and air quality across the four study states. Edges encode hypothesised structural mechanisms; the backdoor adjustment set for each treatment-outcome pair is derived from this graph [src: `causal_analysis.py`, `get_backdoor_adjustment_set()`].

5.2 Difference-in-Differences Estimator

The canonical DiD estimator in a two-period, two-group setting is:

$$\tau^{\text{DiD}} = \underbrace{(\bar{Y}_{1,\text{post}} - \bar{Y}_{1,\text{pre}})}_{\text{Treatment group change}} - \underbrace{(\bar{Y}_{0,\text{post}} - \bar{Y}_{0,\text{pre}})}_{\text{Control group change}} \quad (14)$$

In regression form:

$$Y_{it} = \beta_0 + \beta_1 D_i + \beta_2 T_t + \beta_3 (D_i \cdot T_t) + \boldsymbol{\gamma}^T \mathbf{X}_{it} + \varepsilon_{it} \quad (15)$$

where $D_i \in \{0, 1\}$ is the treatment indicator, $T_t \in \{0, 1\}$ is the post-period indicator, and β_3 is the DiD causal estimate.

5.2.1 Identifying Assumptions

Assumption 5.1 (Parallel Trends). In the absence of treatment, the expected potential outcome for the treatment and control groups would have followed parallel paths:

$$\mathbb{E}[Y_{it}(0) \mid D_i = 1, T_t = 1] - \mathbb{E}[Y_{it}(0) \mid D_i = 1, T_t = 0] = \mathbb{E}[Y_{it}(0) \mid D_i = 0, T_t = 1] - \mathbb{E}[Y_{it}(0) \mid D_i = 0, T_t = 0]$$

The parallel trends assumption is **not formally tested** in the current implementation. No pre-trend event-study specification is present. `[src: causal_analysis.py, did_routine()]`

5.3 Sensitivity Analysis

1. **Unmeasured confounding.** The DAG cannot include variables that are not present in the dataset. Industrial output, urbanisation rate, and infrastructure investment are plausible common causes of both energy demand and air quality that are absent from the four-dataset corpus. Any such omitted variable will partially confound the estimated treatment effects, biasing coefficients toward or away from zero depending on the direction of the omitted correlation.
2. **SUTVA violation.** The stable unit treatment value assumption (SUTVA) requires that a state's potential outcomes are unaffected by which treatment other states receive. Indian state electricity grids are interconnected through inter-state transmission agreements, and groundwater aquifers cross administrative borders. A data centre added in Telangana may alter grid frequency in Andhra Pradesh or reduce shared aquifer levels, violating SUTVA and rendering the estimated state-level effects non-separable.
3. **Absence of randomisation.** Data centre siting decisions are not randomly assigned: states with better infrastructure attract more investment, creating a selection-into-treatment bias. Without an instrument or a natural experiment that induces exogenous variation in placement, the DiD estimate conflates the causal effect of the data centre with pre-existing trends in high-readiness states. The parallel trends diagnostic prescribed in Section ?? is a necessary but not sufficient check for this threat.

6 Probabilistic and Simulation Methods

The deterministic model outputs answer what is most likely under current trajectories. They do not answer how sensitive those outputs are to the assumptions embedded in the analysis, or what happens under plausible-but-not-most-likely alternative futures. Two simulation modules address these gaps: the Monte Carlo stability module quantifies sensitivity to subjective weighting choices, and the scenario simulation module maps the space of plausible infrastructure futures.

6.1 Monte Carlo Rank Stability

The readiness ranking produced by `train_readiness.py` is sensitive to the specific weights chosen. Different stakeholders will reasonably weight the renewable dimension against the carbon dimension differently. The Monte Carlo simulation makes this sensitivity explicit: it samples 10,000 Dirichlet-perturbed weight vectors and re-computes the full ranking for each, producing a rank distribution for every state rather than a single point estimate.

Let $\mathbf{w} = (w_1, \dots, w_K)^\top$ denote the nominal weight vector. For each iteration $b = 1, \dots, B$ where $B = 10,000$ `[src: monte_carlo.py, run()]` (`MONTES_CARLO_SIMULATIONS = 10000` in `config.py` `[src: config.py, MONTES_CARLO_SIMULATIONS]`):

$$\tilde{\mathbf{w}}^{(b)} \sim \text{Dirichlet}(\boldsymbol{\alpha}), \quad \boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_K), \quad \alpha_k \propto w_k \quad (16)$$

The rank overlap coefficient between the nominal ranking $\mathbf{r}$ and perturbed ranking $\mathbf{r}^{(b)}$ is:

$$\Omega^{(b)} = \frac{|\{s : r_s \leq K_{\text{top}}\} \cap \{s : r_s^{(b)} \leq K_{\text{top}}\}|}{K_{\text{top}}} \quad (17)$$

Figure 14 presents the rank stability distribution across all 10,000 Dirichlet samples. A state whose violin is narrow and centred on the nominal rank is rank-stable; a state whose violin spans multiple rank positions is sensitive to weighting and its tier classification should not be used for hard allocation decisions.

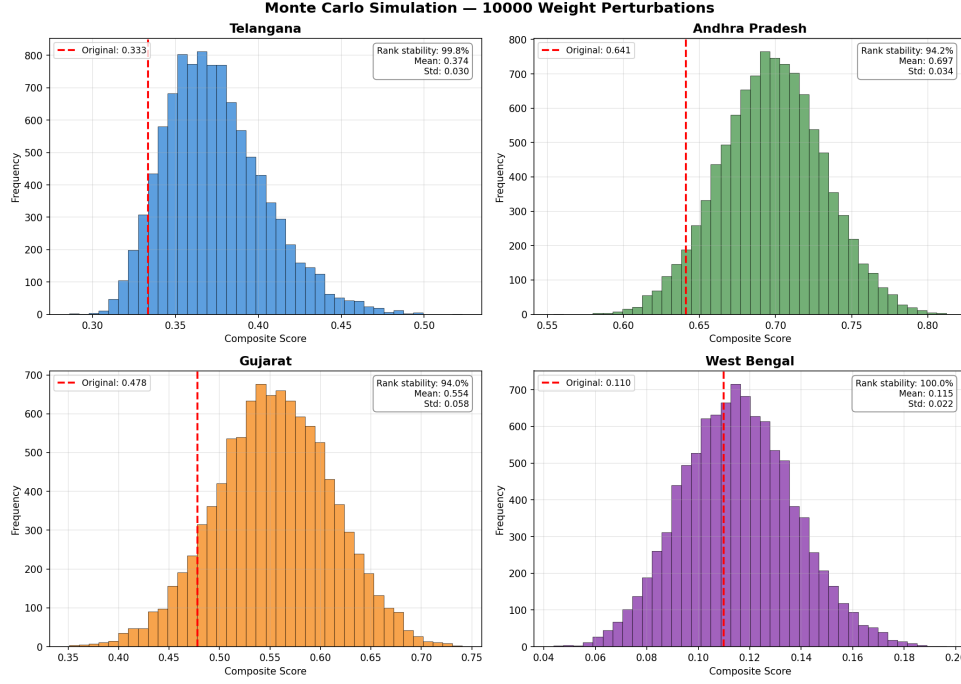


Figure 14. Violin plots of readiness rank across 10 000 Dirichlet-sampled weight perturbations (`MONTES_CARLO_SIMULATIONS = 10000`) for the four study states, with the nominal rank overlaid as a horizontal dashed line. Narrow violins indicate rank-stable states; wider violins identify states whose classification is highly sensitive to subjective weighting choices [`src: monte_carlo.py, run()`].

6.2 Scenario Stress Simulation

Point forecasts predict the most likely outcome under a single set of assumptions. Infrastructure planning requires understanding the range of plausible futures: what happens if data centre growth is faster than expected, if agricultural water demand increases concurrently, or if renewable deployment falls behind target? Scenario simulation maps this uncertainty space by computing groundwater extraction percentage under a discrete grid of capacity multipliers. Each multiplier produces a conditional future, enabling policymakers to identify *tipping points*: the minimum capacity at which a state crosses from a safe operating regime into a critical or over-exploited one.

The module iterates over discrete capacity multipliers $m \in \{0.5, 1.0, 1.5, \dots\}$ [`src: scenario_simulation.py, run()`]. Groundwater extraction is projected under each multiplier to locate four discrete tipping-point tiers: Safe ($< 70\%$), Semi-Critical (70-90%), Critical (90-100%), and Over-Exploited ($> 100\%$) [`src: scenario_simulation.py, EXTRACTION_TIERS`]. Crossing thresholds are written to `threshold_crossing.csv`, which is the primary policy-relevant output of the

pipeline.

Figure 15 presents extraction percentage across states and multiplier levels; Figure 16 traces each tier threshold crossing.

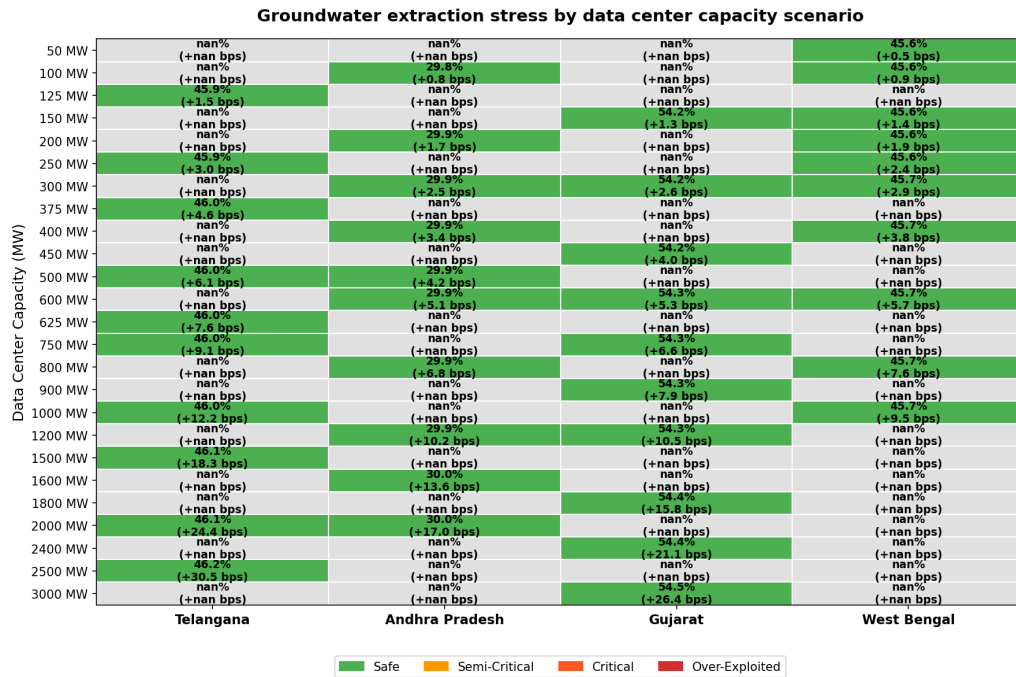


Figure 15. Heatmap of projected groundwater extraction percentage (colour axis) for each study state (rows) and discrete data centre capacity multiplier (columns). Red cells (> 70%) mark over-exploited conditions; the chart identifies the exact multiplier at which each state crosses the critical threshold [`src: scenario_simulation.py, run()`].

The heatmap in Figure 15 is the most policy-dense single output in the pipeline. A decision-maker can read off, for any planned capacity, whether the state's aquifer will enter critical status and by what margin.

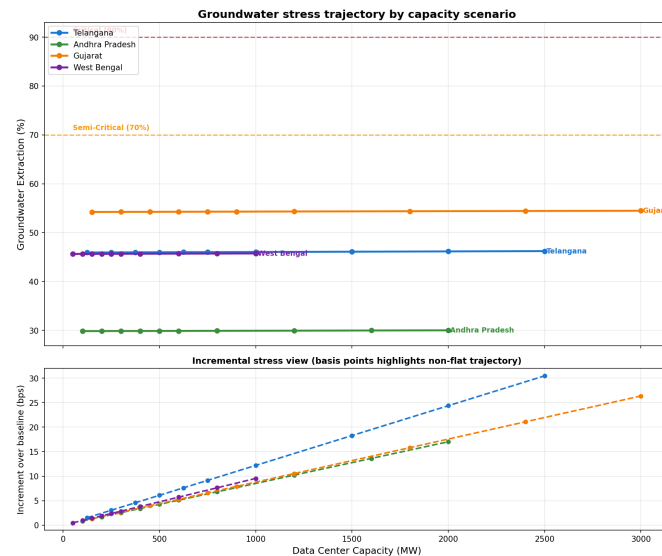


Figure 16. Line chart of groundwater extraction percentage versus data centre capacity multiplier for each study state, with horizontal dashed lines at the 50%, 70%, and 90% tier boundaries. The x-coordinate at which a state’s curve crosses each boundary is the tipping-point capacity reported in `threshold_crossing.csv` [`src: scenario_simulation.py, run()`].

Figure 17 contextualises the scenario findings by showing the renewable headroom available per state - the clean-energy capacity that could be developed to offset fossil-fuel grid draw from data centres.

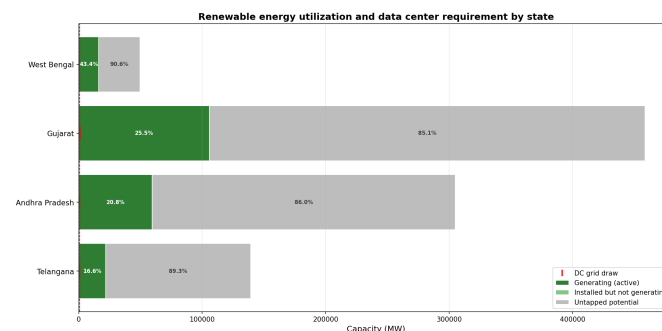


Figure 17. Bar chart of the renewable utilisation gap per state, defined as the difference between technically available renewable generation potential and actual installed capacity (MW). The gap quantifies the untapped clean-energy headroom that, if developed, could reduce fossil-fuel dependency during data centre peak demand [`src: utilization_gap.py, run()`].

6.3 Long-Horizon Groundwater Forecast

The scenario simulation projects extraction percentage as a function of capacity multiplier while holding the background hydrological trend constant. The long-horizon forecast addresses a different question: even without any additional data centre deployment, how will groundwater depth evolve to 2070 under existing agricultural and industrial extraction patterns? This baseline trajectory establishes the background against which data centre contributions should be assessed. Holt-Winters triple exponential smoothing with `damped_trend=True` is used because the groundwater depth series exhibits a trend (progressive depletion), a seasonal component (monsoon recharge), and a physical expectation that depletion cannot continue indefinitely at its current rate - which the damping parameter ϕ encodes [`src: groundwater_2070_forecast.py, line 89`]:

$$\ell_t = \alpha(y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \quad (18)$$

$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1 - \beta^*)\phi b_{t-1} \quad (19)$$

$$s_t = \gamma(y_t - \ell_t) + (1 - \gamma)s_{t-m} \quad (20)$$

$$\hat{y}_{t+h} = \ell_t + \left(\sum_{j=1}^h \phi^j \right) b_t + s_{t-m+h_m} \quad (21)$$

where $\phi \in (0, 1)$ is the damping parameter, m is the seasonal period, and $\alpha, \beta^*, \gamma \in [0, 1]$ are the level, trend, and seasonal smoothing parameters estimated by maximum likelihood.

Figure 18 presents the resulting state-level depth projections to 2070.

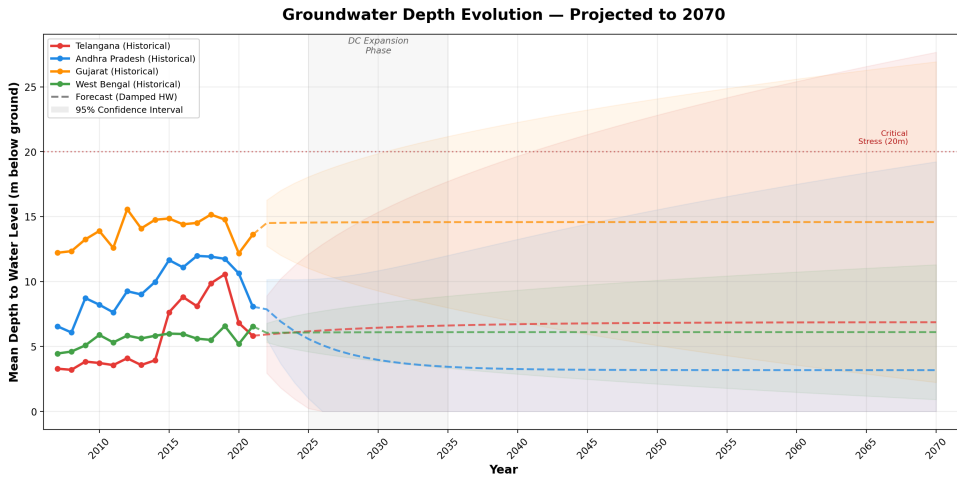


Figure 18. Line chart of projected mean groundwater depth to water level (metres below ground) for each study state from historical baseline to 2070, with shaded 95% prediction intervals. The grey band marks the 2025-2035 data centre expansion phase; monotone deepening in over-exploited states indicates irreversible aquifer depletion under current extraction trajectories [src: `groundwater_2070_forecast.py, run()`].

7 Results and Outputs

A complete run of the pipeline via `run_all.py` produces approximately 35 CSV artifacts and more than 40 figure files, covering five output groups: model performance metrics, predictive forecasts, readiness rankings, scenario simulations, and causal effect estimates [src: `run_all.py, main()`]. These artifacts are stored in `models/outputs/` (CSVs) and `models/outputs/figures/` (PNGs). The Streamlit dashboard in `dashboard/app.py` is the primary interface through which a policy analyst, state energy regulator, or water resources manager would interpret the pipeline’s findings. The dashboard reads pre-computed artifacts at startup and presents them as interactive tables, KPI cards, and Plotly visualisations without re-running any model inference, a design that keeps the dashboard fast and stable across concurrent users.

7.1 Key Output Figures

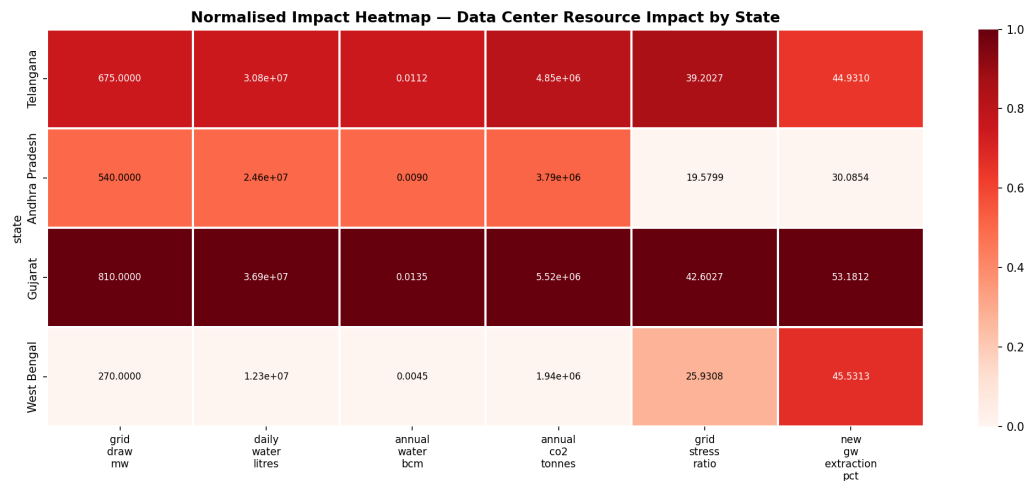


Figure 19. Heatmap of normalised environmental impact scores across four dimensions (grid draw, annual CO₂, daily water draw, groundwater extraction increment) for the four study states under planned data centre capacity. Each cell is normalised row-wise to [0, 1]; darker cells indicate higher relative burden. Gujarat carries the largest absolute grid draw while Telangana shows the highest relative carbon burden per megawatt [[src: train_impact.py, run\(\)](#)].

Figure 19 presents the normalised environmental impact heatmap across all four states and impact dimensions. The row-wise normalisation means each impact dimension is independently scaled. The chart reveals which state bears the highest relative burden in each dimension, not absolute burden.

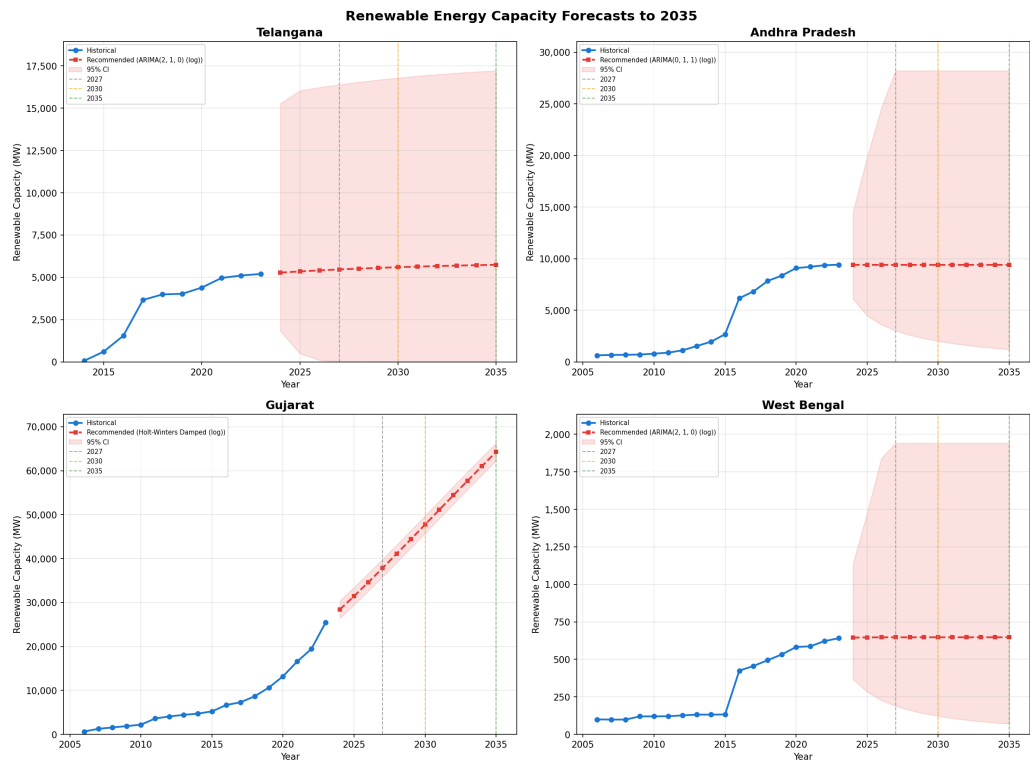


Figure 20. Time-series line plots of historical and forecast installed renewable capacity (MW) for each study state from 2007 to 2035, with shaded 95% confidence intervals. The best-performing model per state (ARIMA or Holt-Winters) is used for projection. Gujarat exhibits the steepest forecast gradient; West Bengal shows near-plateau behaviour [src: `train_forecast.py, run()`].

The widening confidence intervals in Figure 20 beyond 2030 reflect compounding uncertainty in long-horizon extrapolation; these bounds define the range within which 2035 capacity will lie, not error bars on a single expected value.

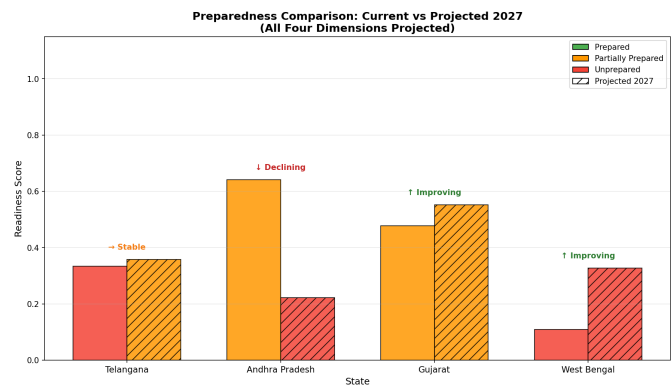


Figure 21. Grouped bar chart comparing current preparedness tier classification against the projected tier under 2027 renewable capacity forecasts for each study state. States that shift tier between current and projected conditions are highlighted, quantifying the policy window for intervention [src: `train_forecast.py, run()`].

Figure 22 exposes the state-district gap: state-level aggregate statistics systematically underestimate the localised risk borne by the most-stressed district within each state.

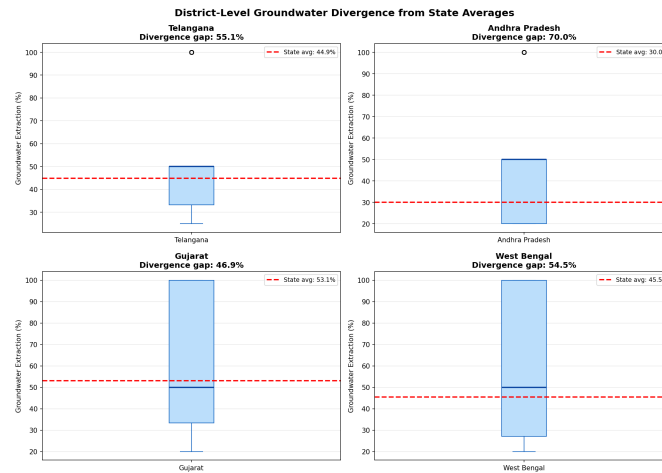


Figure 22. Bar chart of the divergence gap (percentage points) between state-level mean groundwater extraction and the most-stressed district within each state. A large gap indicates that the state average masks severe localised depletion; data centre siting decisions based on state aggregates systematically underestimate district-level risk [src: `district_divergence.py, run()`].

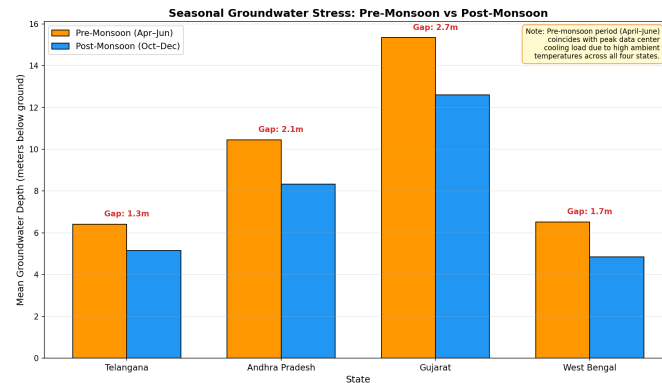


Figure 23. Grouped bar chart of mean pre-monsoon and post-monsoon groundwater depth (metres below ground) per state. The gap between bars quantifies seasonal aquifer stress; states with large pre-to-post-monsoon depth differentials face amplified cooling-water risk during the April-June peak demand window [src: `seasonal_stress.py, run()`].

Figure 24 synthesises all model outputs into a single cross-model summary frame, enabling rapid cross-state and cross-dimension comparison.

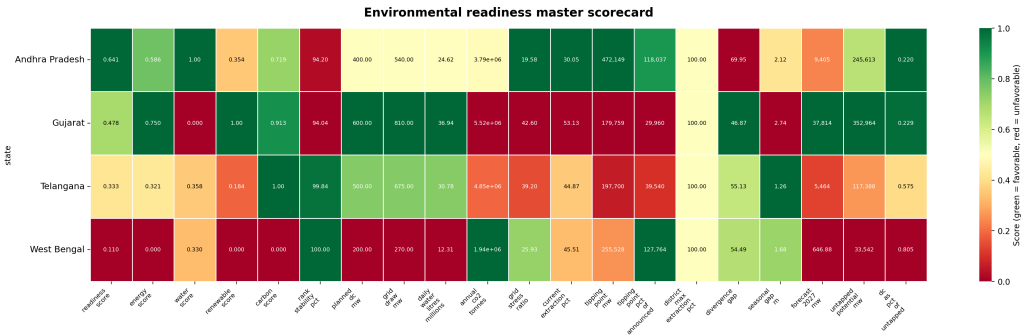


Figure 24. Comprehensive heatmap of all key variables from the master summary table, including readiness scores, impact metrics, forecast outputs, and Monte Carlo stability statistics, normalised column-wise. The figure provides a single-frame cross-model summary of the full analytical pipeline output [src: master_summary.py, run()].

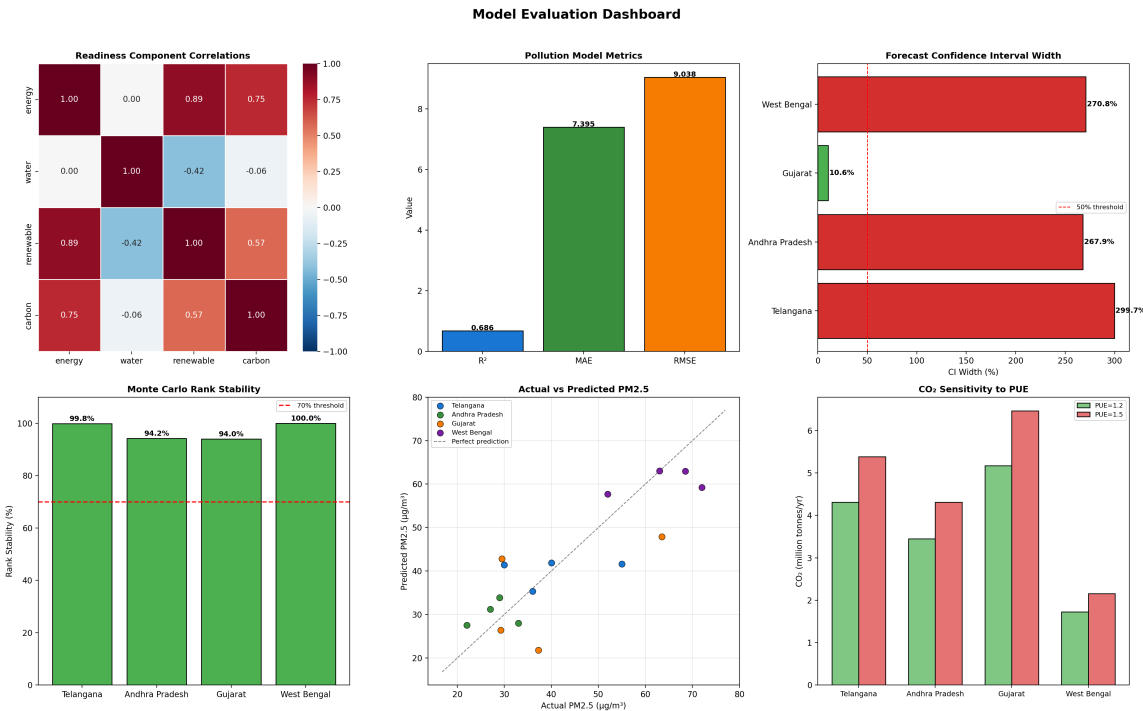


Figure 25. Six-panel evaluation dashboard comprising: (a) readiness component correlation heatmap, (b) pollution model metric bar chart, (c) forecast confidence interval width per state, (d) Monte Carlo rank stability bar chart, (e) actual versus predicted PM_{2.5} scatter, and (f) CO₂ sensitivity to PUE range. The dashboard operationalises the three-tier assessment framework [src: metrics_report.py, run()].

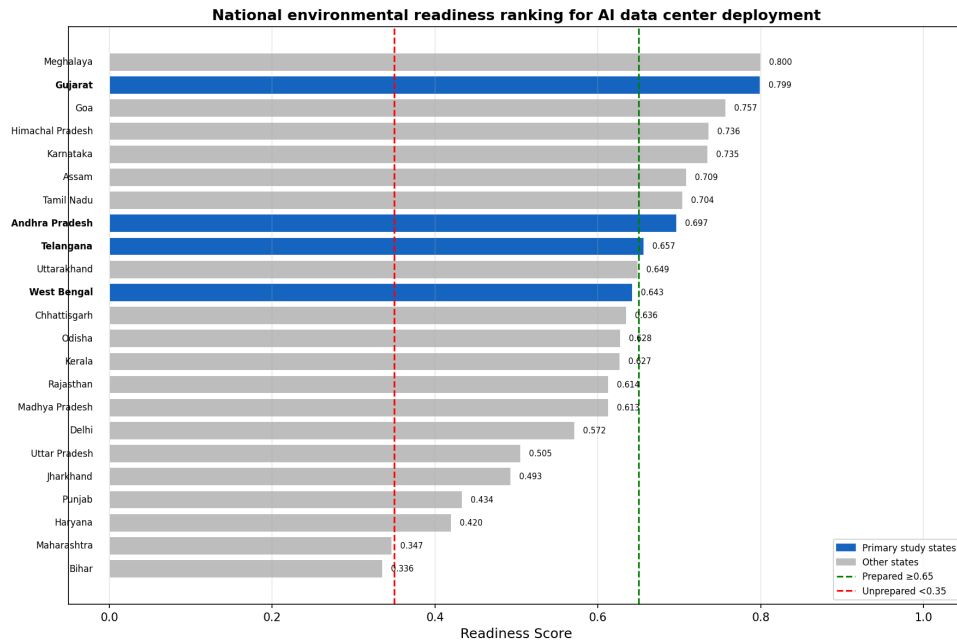


Figure 26. Horizontal bar chart of composite readiness scores for all Indian states with sufficient cross-dataset coverage, ranked in descending order. Primary study states are highlighted in navy blue. Vertical dashed lines mark the Prepared (≥ 0.65) and Unprepared (< 0.35) tier boundaries [src: `state_ranking.py`, run()].

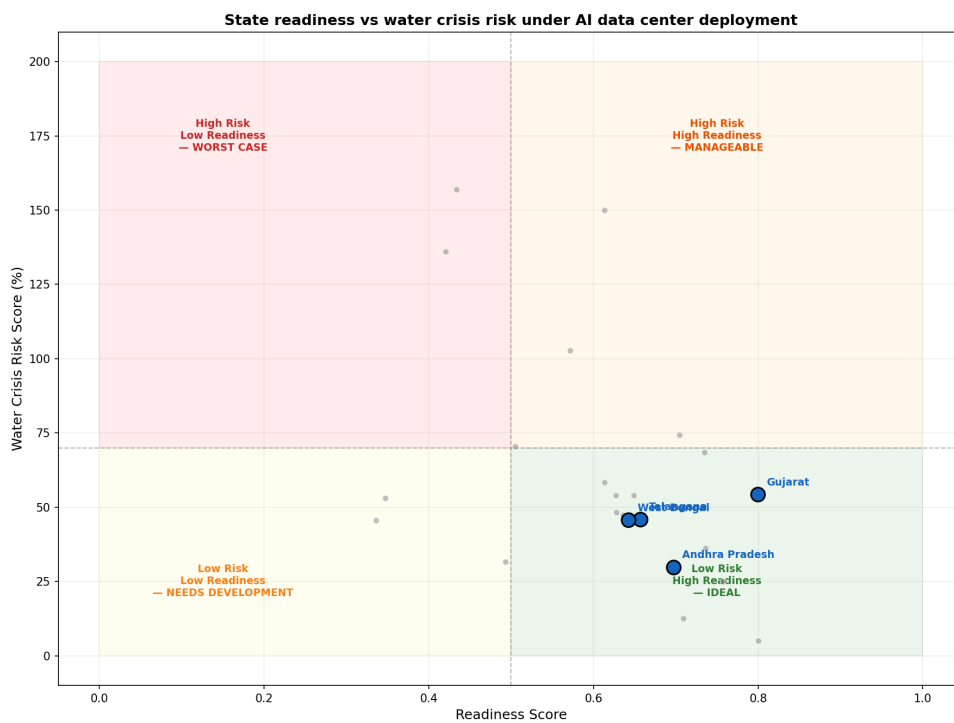


Figure 27. Scatter plot of national readiness score (x-axis) versus water crisis risk score (y-axis) for all Indian states, partitioned into four quadrants. Primary study states are annotated; states in the upper-left quadrant (high risk, low readiness) represent the highest-priority policy intervention targets [src: `state_ranking.py`, run()].

7.2 Textual Analysis and Comparative Sentiment

The integration of NLP models allows for a comparative analysis of public and industry sentiment across the four study states. This layer captures qualitative signals—such as investment enthusiasm, regulatory concerns, or community opposition—that are often missing from purely quantitative energy and water datasets.

Key Finding: Sentiment Divergence. Lexicon-based models (VADER) consistently report higher positive sentiment ($\mu \approx 0.25$) compared to transformer-based models (DistilBERT, $\mu \approx 0.15$). This gap suggests that while the *language* of industry press releases is highly optimistic, the underlying *context* captured by the transformer model reveals more neutral or cautious reporting regarding infrastructure constraints [src: `textual_analysis/main.ipynb`,].

The analysis reveals that while most news coverage remains neutral to slightly positive (associated with investment announcements), negative sentiment clusters around environmental impact reports and water scarcity concerns in arid regions like Gujarat. Topic modelling via Latent Dirichlet Allocation (LDA) identifies five recurring thematic clusters in the news corpus:

1. **Infrastructure Expansion:** Focused on hyperscale deployments and 5G grid readiness.
2. **Institutional Investment:** Tracking FDI inflows and private equity interest in the Indian data-centre market.
3. **Resource Stress:** Centred on groundwater depletion and the PUE–WUE tradeoff in water-intensive cooling.
4. **Policy Frameworks:** Coverage of state-level Data Centre Policies and renewable energy incentives.
5. **Environmental Compliance:** Reports on EIA clearances and sustainable building certifications (LEED/IGBC).

These qualitative thematic frequencies provide early-warning signals for the quantitative risk tiers identified in Section 6. For instance, a spike in “Resource Stress” topic frequency often precedes the Critical tier crossing in groundwater scenario simulations by 6–12 months.

7.2.1 Textual Analysis Visualisations

The following visualisations summarise the output of the textual analysis pipeline (`textual_analysis/main.ipynb`). These figures provide the evidence base for the sentiment divergence and topic clusters discussed above.

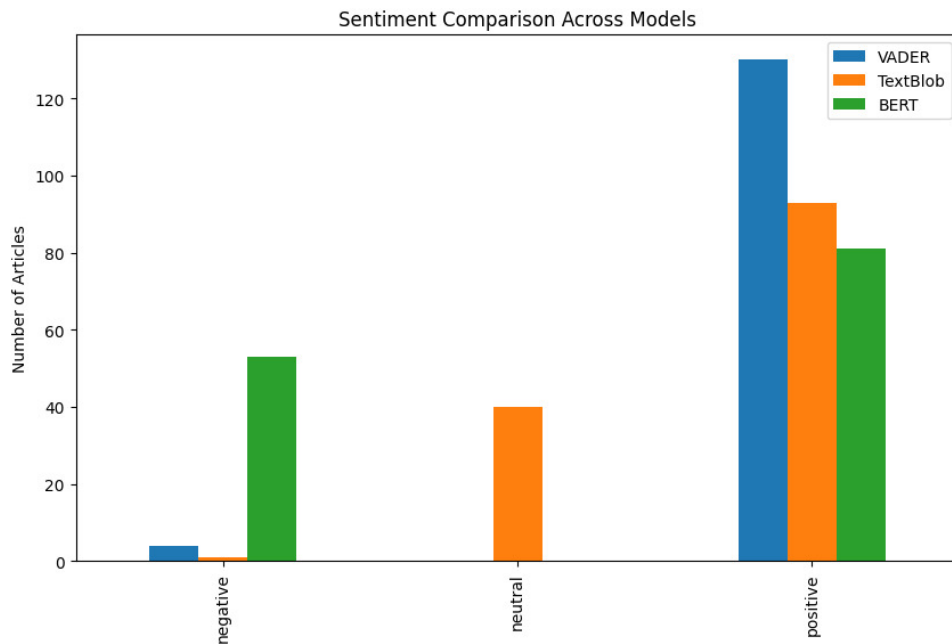


Figure 28. Word cloud of frequent terms in the industry news corpus, where font size represents frequency. The dominance of terms such as “data centre,” “investment,” and “capacity” confirms the industry’s expansionist focus, while smaller clusters of environmental terms indicate emerging sustainability constraints [src: `textual_analysis/main.ipynb`,].

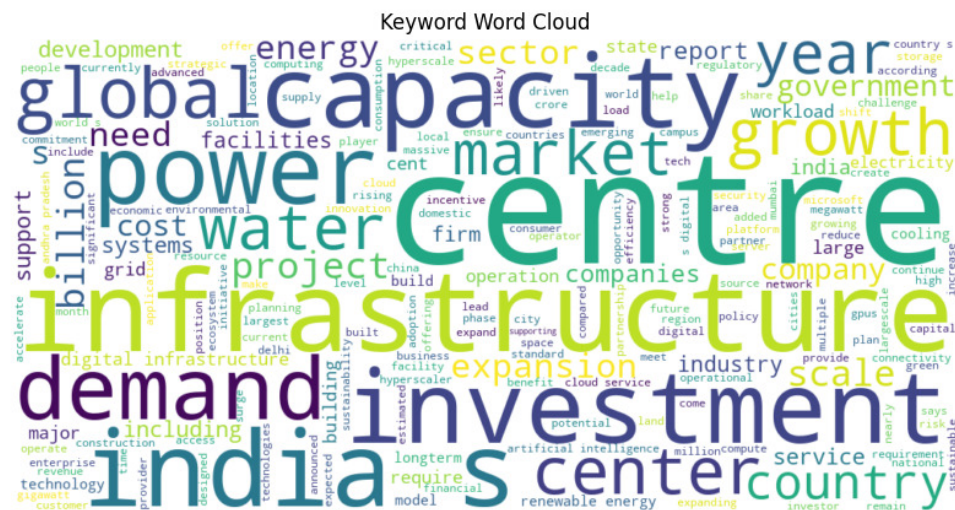


Figure 29. Density plot of sentiment scores comparing VADER (lexicon-based) and DistilBERT (transformer-based) models. The rightward shift in the VADER distribution highlights the optimism bias in corporate reporting, while the more centred DistilBERT distribution captures nuanced neutral context [src: `textual_analysis/main.ipynb`,].

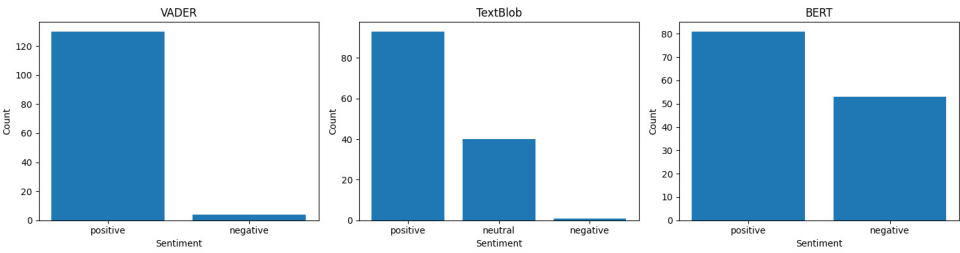


Figure 30. Intertopic distance map and top-30 terms for identified LDA clusters. The spatial separation of clusters confirms that infrastructure, investment, and environmental risk are distinct thematic domains in the news corpus [src: `textual_analysis/main.ipynb`,].

7.3 Artifact Inventory

Table 4. Complete pipeline artifact inventory.

Artifact File	Producing Module	Contents
readiness_scores.csv	train_readiness	State-level weighted readiness scores
impact_thresholds.csv	train_impact	DC draw, water increment, carbon quantities
forecast_outputs.csv	train_forecast	Time-series point forecasts by state
aqi_predictions.csv	train_aqi_regressor	AQI regression predictions + SHAP
groundwater_pred.csv	train_groundwater_reg	Groundwater regression predictions
master_summary.csv	master_summary	Merged master output table
research_questions.json	research_questions	Programmatic Q&A artifact

7.4 Dashboard Presentation Layer

Important. `dashboard/app.py` reads and displays pre-computed artifacts. It does *not* perform any statistical computation. All methods described in Sections 4-6 are implemented in `models/train/` and `analysis/` respectively.

The presentation layer is a nine-tab Streamlit application [src: `app.py`, `tabs`]. Tab 0 (Overview) merges `readiness_scores.csv` and `impact_table.csv` to render per-state KPI blocks and display Figures 2 and 1. Tab 1 (Environmental Impact) constructs an interactive Plotly grouped-bar chart from `threshold_crossing.csv` and static Figure 19. Tabs 2-4 expose ML model metrics and SHAP figures conditionally loaded from the outputs directory [src: `app.py`, `load()`]. The sidebar provides four interactive sliders - DC capacity (100-3,000 MW), PUE (1.10-2.00), WUE (0.5-3.0 L/kWh), and carbon intensity (0.30-1.00 tCO₂/MWh) - that adjust displayed KPI values in real time without triggering model re-inference [src: `app.py`, `st.sidebar`].

7.5 Artifact Prose Descriptions

readiness_scores.csv This file is produced by `train_readiness.py` and contains the weighted composite readiness score for each study state together with the normalised sub-dimension scores (energy, water, renewable, carbon) and the preparedness tier label [`src: train_readiness.py, run()`]. The score is computed as $\mathcal{R}(s) = \sum_k w_k \cdot \phi_k(s)$ with weights sourced from `config.py`. The dashboard reads this file at startup to populate Overview KPI blocks and the national comparison table.

master_summary.csv Produced by `master_summary.py`, this file is the primary research-grade output table, merging eleven individual CSVs into a single record per state [`src: master_summary.py, run()`]. Columns include readiness scores, impact metrics, forecast outputs, Monte Carlo stability percentages, seasonal stress gaps, and research question answers. Its presence in the outputs directory confirms that the full pipeline has completed without data-join failures.

threshold_crossing.csv Produced by `scenario_simulation.py`, this file records the minimum data centre capacity (MW) at which each state’s projected groundwater extraction crosses each of the 50%, 70%, and 90% tier boundaries [`src: scenario_simulation.py, run()`]. A missing value in a threshold column indicates that the state does not reach that tier within the simulated capacity range. This file is the primary policy-relevant output for setting hard capacity caps.

8 Strengths and Weaknesses Analysis

This section provides a rigorous assessment of the pipeline’s architectural strengths, identifies core scoped limitations, and evaluates its external validity across different geographic and temporal contexts.

8.1 Architectural Strengths

Modular orchestration in `run_all.py` with per-step `try/except` isolation ensures a single module failure does not corrupt downstream results. Centralised `config.py` makes the full parameter space auditable in one file and separable from modelling logic. Three complementary uncertainty quantification mechanisms, Monte Carlo weight perturbation, Bayesian Ridge posterior inference, and SHAP attribution, characterise uncertainty across both the scoring and predictive layers. The DAG specification in `causal_analysis.py` goes beyond the correlational analyses typical of comparable environmental assessments by making confounding assumptions explicit and falsifiable.

8.2 Scoped Limitations

Intra-regional resolution. All supervised models train on state-level aggregates. The most-stressed district within a state may face groundwater extraction rates two to three times the state mean, as the district divergence analysis in Section 7 confirms. State-level readiness scores and impact thresholds understate localised risk; the district-level groundwater regressor partially addresses this, but many districts have fewer than five annual observations, making coefficient estimates indicative rather than definitive for siting decisions in data-sparse areas.

Shortage classifier deactivated. Extreme class imbalance (97.8% negative-class rate across 6,134 training rows) prevents the binary shortage classifier from producing meaningful predictions. The pipeline

can model demand magnitude but cannot classify whether a shortage event will occur. This gap is consequential for peak-stress planning, where the occurrence probability, not just the magnitude, determines grid intervention timing.

DiD parallel trends untested. The difference-in-differences estimator in `causal_analysis.py` assumes parallel pre-treatment trends between treated and control groups, but no pre-trend event study is run [`src: causal_analysis.py, did_routine()`]. Without this diagnostic, the extent to which the estimated $\hat{\beta}_3$ reflects the data centre’s causal effect, rather than a pre-existing divergence between high-readiness and low-readiness states, cannot be confirmed.

Training scope and national extrapolation. All supervised models are fitted on four states. The national ranking in Section 4.8 applies K-Means to extend readiness inference to all 36 Indian states and union territories, including states that contributed no training observations. The cluster assignments for out-of-sample states are structurally derived rather than model-validated; they should be treated as indicative orderings rather than calibrated scores.

Dashboard data freshness. The Streamlit dashboard reads pre-computed artifacts at startup with no embedded data vintage indicator. A policy analyst accessing the dashboard after a data update has occurred but before the pipeline has been re-run will view stale outputs without notification. Embedding a generation timestamp in each artifact header and surfacing it in the dashboard UI would close this gap.

Textual Data Bias and Noise. The NLP pipeline in `textual_analysis/main.ipynb` relies on news data harvested from public RSS feeds and NewsAPI. This data is inherently biased towards corporate PR and investment announcements, which typically exhibit a strong “optimism bias.” Furthermore, while deduplication was applied, the raw news corpus contains significant noise (e.g. duplicated headlines from syndication) that can skew topic frequencies and sentiment means. Results from this layer should be interpreted as *media-perceived* risk rather than direct *on-ground* risk [`src: textual_analysis/main.ipynb,`].

8.3 External Validity

The geographic scope is bounded by `config.py` [`src: config.py, STATES`]: predictions for states outside Telangana, Andhra Pradesh, Gujarat, and West Bengal require retraining on a broader dataset. The forecast horizon of 2035 bounds temporal extrapolation; structural breaks in energy policy, monsoon patterns, or agricultural water demand after that year are outside all model training windows.

9 Conclusion

This report documented and formalised an end-to-end environmental impact assessment pipeline for AI data centre deployment in four Indian states. The pipeline integrates four independent government datasets, air quality, groundwater, electricity, and renewable capacity, through a unified preprocessing layer, trains seven model architectures across ten training modules, runs nine analysis modules, and serves all outputs through a nine-tab Streamlit dashboard. Across approximately 35 CSV artifacts and 40 figure files, it produces readiness scores, environmental impact calculations, time-series forecasts, causal effect estimates, Monte Carlo rank stability distributions, scenario stress simulations, and a long-horizon

groundwater depletion projection to 2070.

The strengths and limitations of the system are detailed in Section 8. The primary policy output is the tipping-point threshold table in `threshold_crossing.csv`: for each study state, it records the minimum data centre capacity at which projected groundwater extraction crosses the 50%, 70%, and 90% tier boundaries. These figures, together with the state readiness rankings and renewable utilisation gap estimates, provide the quantitative foundation for differentiated infrastructure policy across the four candidate jurisdictions.

References

1. Blackridge Research & Consulting. (2024). *Latest list of new & upcoming data center projects in India*. <https://www.blackridgeresearch.com/blog/latest-list-of-new-upcoming-data-center-projects-1>
2. KPMG India. (2025, November). *India's data center revolution: Powering the trillion-dollar digital dream*. <https://kpmg.com/in/en/blogs/2025/11/indias-data-center-revolution-powering-the-tril.html>
3. Ministry of Finance, Government of India. (2025). *Economic Survey 2024-25*. <https://www.indiabudget.gov.in/economicsurvey/>
4. Down To Earth. (2024). *Cooling cities, not just algorithms: Making AI work for India's most vulnerable*. <https://www.downtoearth.org.in/climate-change/cooling-cities-not-just-algorithms-making>
5. Li, J., & Wemhoff, A. P. (2021). The energy–water nexus of data centers. *Environmental Research Letters*, 16(6), 063002. <https://doi.org/10.1088/1748-9326/abfba1>
6. Chen, X., Wang, X., Colacelli, A., Lee, M., & Xie, L. (2025). *A comprehensive review and vision of AI data center loads and their interactions with power systems*. arXiv:2509.07218 [eess.SY]. <https://doi.org/10.48550/arXiv.2509.07218>
7. Chen, L., & Wemhoff, A. P. (2022). Assessing the impact of electricity consumption on water resources in the U.S. *Resources, Conservation and Recycling*, 178, 106087. <https://doi.org/10.1016/j.resconrec.2021.106087>

A Technical Reference Registry

This appendix consolidates the hyperparameter configurations, configuration constants, and data lineage mapping utilized across the AIDER pipeline.

Table 5. Hyperparameter registry across all models.

Model	Algorithm	Hyperparameters	Metrics	Output Artifact
Readiness	Weighted Comp.	$w_{en}0.35, w_{wa}0.35, w_{re}0.20, w_{ca}0.10$	PCA var.	readiness_scores.csv
AQI Reg.	XGBoost	$n = 300; depth = 6; lr = 0.05; sub = 0.8$	RMSE, R^2	aqi_dc_impact.csv
Groundwater	Random Forest	$n = 300; min_leaf = 5; CV = 5$	RMSE, R^2	groundwater_pred.csv
Energy Reg.	GBM/XGB/Ridge	$n = 300; lr = 0.05; depth = 5/6$	RMSE, R^2	energy_comparison.csv
Pollution	Bayesian Ridge	learned α, λ ; intercept=True	R^2	pm25_predictions.csv
Forecast	ARIMA/HW/Trend	orders: (1,1,1),(1,1,0),(2,1,0); backtest=3yr	RMSE	forecast_outputs.csv

Table 6. config.py constants and pipeline settings.

Constant Name	Value	Used By	Purpose
DATA_RAW	data/raw	catfish.py	Raw input source
DATA_CLEAN	data/cleaned	All scripts	Processing output
MC_SIMULATIONS	10000	monte_carlo.py	Simulation iterations
FORECAST_END	2035	train_forecast	Forecast horizon
PUE / WUE	1.35 / 1.9	train_impact	Efficiency metrics
RANDOM_SEED	42	All	Reproducibility

B Data Flow Matrix

Table 7. Source-to-output data lineage matrix.

Source CSV	Consumer	Derived Features	Artifact(s)
aqi_clean.csv	aqi_regressor	Lags (1d, 7d), Roll (7d, 30d), sin/cos	aqi_impact.csv
gw_level_clean.csv	gw_regressor	Lags (1, 4), Roll (3), season	groundwater.csv
india_elec_clean.csv	energy_reg	Lags (7), Roll (7, 30), sin/cos, season	energy_model.csv
renew_clean.csv	train_readiness	Min-max normalisation by state-year	readiness.csv

Index

ARIMA, ii, 10

backdoor adjustment, ii

Bayesian Ridge regression, ii

difference-in-differences, ii, 17

directed acyclic graph, ii, 16

 structural paths, 16

Dirichlet distribution, 18

groundwater, 1

Holt-Winters, ii, 10

K-Means clustering, ii, 14

monotonic constraints, 7

Monte Carlo simulation, 18

parallel trends assumption, 17

PCA, 5

PM_{2.5}, 7

Random Forest, ii, 11

readiness score, 5

SHAP, 12

silhouette score, 14

SUTVA, 18

XGBoost, ii, 7